COMFYUI · VAST.AI · RTX 5090 / 4090

Guía de paso a producción

De cero a ComfyUI funcionando en una GPU alquilada, en una tarde. Imagen Docker "Everything-Ready" publicada en GHCR, alquilada con el helper `comfy`.

Versión 1.0 · Repositorio ComfyUIPod · Rama `main` · HEAD `5e74d80`
Generada a partir del estado verificado del repositorio (README, decisiones, walkthrough, workflows y helper).

A. Qué tienes ahora

Tienes una imagen Docker lista para producción que convierte cualquier GPU RTX 5090 o RTX 4090 alquilada en Vast.ai en un ComfyUI completo y privado, accesible 100% desde el navegador con usuario y contraseña. La imagen incluye ComfyUI con 10 custom nodes preinstalados, 63.35 GiB de modelos que se descargan automáticamente en dos fases (los ligeros antes de abrir, los pesados en segundo plano mientras trabajas), un portal de bienvenida en `/home`, un panel de estado en vivo en `/status` con métricas de GPU y diagnóstico automático, un supervisor en Rust (`comfyctl`) que verifica la máquina antes de gastar dinero en descargas, y varios perfiles de modelos (`all`, `general`, `realistic`, `anime`, `minimal`) para pagar solo por lo que necesitas. GitHub Actions construye y publica la imagen en GHCR, y el script `comfy` para macOS/Linux te deja alquilar, abrir, monitorizar y destruir instancias con un comando cada uno.

B. Checklist de una tarde

1. Crear el repo en GitHub y subir el código

Qué haces: creas un repositorio nuevo (vacío, sin README) en GitHub y empujas el código que ya tienes. Este repo local **aún no tiene remote configurado** (comprobado con `git remote -v`), así que estos comandos aplican tal cual.

```
# Sustituye <tu-usuario> por tu usuario de GitHub
git remote add origin https://github.com/<tu-usuario>/comfyuipod.git
git push -u origin main
```

Cómo sé que va bien: el push termina sin errores y al refrescar la página del repo en GitHub ves tus archivos (`Dockerfile`, `comfyctl/`, `scripts/` …).

2. Esperar el primer workflow verde

Qué haces: en la pestaña **Actions** del repo, el workflow *"Build, Validate and Publish ComfyUI Vast.ai Image (GHCR)"* arranca solo tras el push. Valida los scripts, compila la imagen Docker, le hace un smoke test y la publica en GHCR. La primera vez tarda **entre 60 y 120 minutos (pendiente de medir)**; los siguientes builds reutilizan caché y tardan minutos.

Cómo sé que va bien: el workflow aparece con un ✓ verde. Si falla, abre el job rojo y lee el paso marcado con ✗: cada nombre de paso indica qué se validaba (ShellCheck, Hadolint, SHAs de nodos, disponibilidad de modelos en HuggingFace, tests de Rust…).

3. Hacer público el paquete en GHCR

Qué haces: para que Vast.ai descargue la imagen sin credenciales, ve a tu perfil de GitHub → **Packages** → **comfyuipod** → **Package settings** → **Change visibility** → **Public**. El nombre final de la imagen será:

```
ghcr.io/<tu-usuario>/comfyuipod:latest
```

Confirma el nombre exacto tras el primer push (aparece en la página del paquete; GitHub lo pone en minúsculas).

Cómo sé que va bien: abres en una ventana de incógnito la página del paquete (`https://github.com/<tu-usuario>/comfyuipod/pkgs/container/comfyuipod`) y se ve sin iniciar sesión.

4. Crear cuenta en Vast.ai y añadir saldo

Qué haces: registrarte en vast.ai, verificar la cuenta y cargar saldo. Con **$5–10** te sobra para todas las pruebas de esta guía (una sesión de aceptación de una hora cuesta céntimos).

Cómo sé que va bien: en *Billing* ves tu saldo disponible y en *Account* puedes copiar tu **API Key**.

5. Preparar la Mac (vastai + helper comfy)

Qué haces: instalas la CLI de Vast.ai, la autenticas e instalas el helper:

```
pip install --upgrade vastai
vastai set api-key <TU_VASTAI_API_KEY>

mkdir -p ~/.local/bin
cp tools/mac/comfy ~/.local/bin/comfy
chmod +x ~/.local/bin/comfy
```

Añade a tu `~/.zshrc` si no está ya: `export PATH="$HOME/.local/bin:$PATH"`

Cómo sé que va bien: `comfy all` lista tus instancias (ninguna aún) sin errores de autenticación.

6. Apuntar el helper a tu imagen

```
export COMFY_IMAGE=ghcr.io/<tu-usuario>/comfyuipod:latest
```

Cómo sé que va bien: al lanzar `comfy rent`, el resumen previo a la confirmación muestra la imagen correcta.

7. Primera sesión de aceptación

Qué haces:

```
comfy rent general   # perfil general: 27.75 GiB de modelos, buena prueba equilibrada
comfy open           # espera al arranque y abre el portal /home
comfy report         # descarga /report.md y lo copia al portapapeles
```

El helper te muestra el Top 5 de ofertas RTX 5090/4090 con **el precio por hora real antes de confirmar**: ahí ves exactamente lo que vas a pagar.

Cómo sé que va bien: primero ves la pantalla de espera (Fase 1 descargando los modelos ligeros, ~25–60 s según red); el portal `/home` carga con sus tarjetas; `/status` muestra GPU, VRAM y progreso de descargas; y al cabo de unos minutos ComfyUI abre y puedes arrastrar un workflow. Pulsa **R** en ComfyUI para refrescar los desplegables cuando termine la Fase 2 (~4–8 min a 1 Gbps / 500 Mbps en perfil `general`).

8. Verificaciones clave de aceptación

1. **Sesión real con descargas:** en `/status`, la Fase 2 avanza y los checkpoints del perfil `general` aparecen en ComfyUI tras pulsar **R**.
2. **FATAL simulado:** alquila una instancia con una GPU pequeña (menos de 20 GB de VRAM, p. ej. una RTX 3060): el arranque estricto debe mostrar el **banner rojo FATAL** en `/home` y `/status`, y la instancia saldrá con código **20** pasados 10 minutos de gracia. Ejecuta `comfy destroy <id>` en cuanto lo veas para no pagar.
3. `comfy watch`**:** muestra el estado en vivo desde la terminal cada 3 segundos.
4. **Secretos redactados:** `comfy report --stdout` no muestra tu `COMFY_PASS` ni tokens: `comfyctl` los redacta automáticamente en `/report.md`.
5. **Variables con espacios (cierra V27 / S-07):** prueba `comfy rent` con `COMFYUI_EXTRA_ARGS="--fast --cpu"` exportada y verifica en `/status/api` que `env_masked` la muestra correctamente. Si el entrecomillado se rompe, créala a mano con `vastai create instance`.

9. Al terminar: destruir la instancia

```
comfy destroy <id>
```

Cómo sé que va bien: `comfy all` ya no la lista, o la muestra destruyéndose. **El disco sigue facturando aunque pares la instancia**: solo `destroy` detiene todos los cargos.

C. Costes aproximados

Concepto	Coste aproximado	Notas
RTX 5090 (32 GB)	~$0.30–0.60/h	El precio exacto se ve en `comfy rent` antes de confirmar (varía por oferta)
Disco (~120 GB)	Unos céntimos/hora adicionales	Incluido en la estimación de coste total/hora que muestra `comfy rent`
Sesión de prueba de 1 h	Céntimos a ~$0.60	Según la oferta elegida
Instancia **parada** (no destruida)	El disco **sigue facturando**	Por eso el paso 9 es `destroy`, no "stop"

Regla de oro: máquina encendida solo mientras la usas; `comfy destroy` al terminar.

D. Qué hacer si algo falla

Síntoma	Qué significa	Qué hacer
Banner rojo **FATAL** en `/home` / `/status`	El arranque estricto (`strict-check` S1–S10) detectó un host incompatible (GPU, VRAM, RAM, disco…)	`comfy destroy <id>` y alquila otra oferta; el banner indica causa y remediación. La instancia sale sola con código 20 tras 10 min
Pantalla de espera eterna	La Fase 1 está descargando o algo la bloqueó	Abre `/status` y mira el progreso de descargas y la pestaña de logs; el motor de triaje (17 patrones) suele nombrar la causa exacta
Error 401 al abrir el portal	Contraseña incorrecta o no definida	`comfy pass <id>` recupera la contraseña autogenerada desde los logs de arranque
ComfyUI no responde pero el portal sí	El supervisor está reiniciando `main.py` (OOM, crash CUDA…)	Mira `/status` (clasifica la salida: 137 OOM, 134/139 crash nativo…) y `/report.md` para el informe completo
Faltan modelos en los desplegables	Fase 2 aún en curso	Espera a que termine en `/status` y pulsa **R** en ComfyUI
`comfy rent` falla con la imagen	`COMFY_IMAGE` mal exportada o paquete GHCR privado	Revisa el paso 6 y que el paquete sea público (paso 3)
Algo raro sin nombre claro	—	`comfy report <id>`: el informe está pensado para pegárselo a un agente IA y que lo diagnostique

E. Notas de seguridad básicas

- **HTTP (no HTTPS):** el portal y ComfyUI van por HTTP en el puerto directo de Vast.ai, protegidos solo por Basic Auth. Es un riesgo **aceptado y documentado**: la contraseña se autogenera aleatoria

(24 caracteres hex) y es única por instancia. No la reutilices para nada más; si defines `COMFY_PASS` tú mismo, usa una contraseña exclusiva.

- **No expongas más puertos:** solo el 8188 mapeado está previsto; no publiques otros.
- **Secretos fuera de los logs:** `COMFY_PASS` nunca se imprime si la defines tú, y `comfyctl` redacta secretos automáticamente en `/report.md` y `/status/api`. Aun así, no pegues informes en sitios públicos sin echarles un vistazo.
- **Destruye siempre:** una instancia olvidada es un gasto y una puerta abierta.

F. Checklist final imprimible

☐ 1. Repo en GitHub creado y `git push -u origin main` hecho

☐ 2. Workflow "Build, Validate and Publish" en verde en Actions (60–120 min, pendiente de medir)

☐ 3. Paquete GHCR `comfyuipod` puesto como **Public**

☐ 4. Cuenta Vast.ai creada con $5–10 de saldo

☐ 5. `vastai` instalado con API key y helper `comfy` en `~/.local/bin`

☐ 6. `export COMFY_IMAGE=ghcr.io/<tu-usuario>/comfyuipod:latest`

☐ 7. `comfy rent general` → `comfy open` → pantalla de espera, `/home`, `/status` y ComfyUI OK → `comfy report`

☐ 8. Verificaciones: descargas reales, FATAL simulado con GPU pequeña (código 20), `comfy watch`, secretos redactados en `/report.md`, `COMFYUI_EXTRA_ARGS="--fast --cpu"` (V27/S-07)

☐ 9. `comfy destroy <id>` ejecutado — instancia y cargos detenidos

Generada a partir del estado verificado del repositorio en `main` (HEAD `5e74d80`): README.md, docs/decisiones.md, docs/walkthrough.md, .github/workflows/docker-publish.yml y tools/mac/README.md.